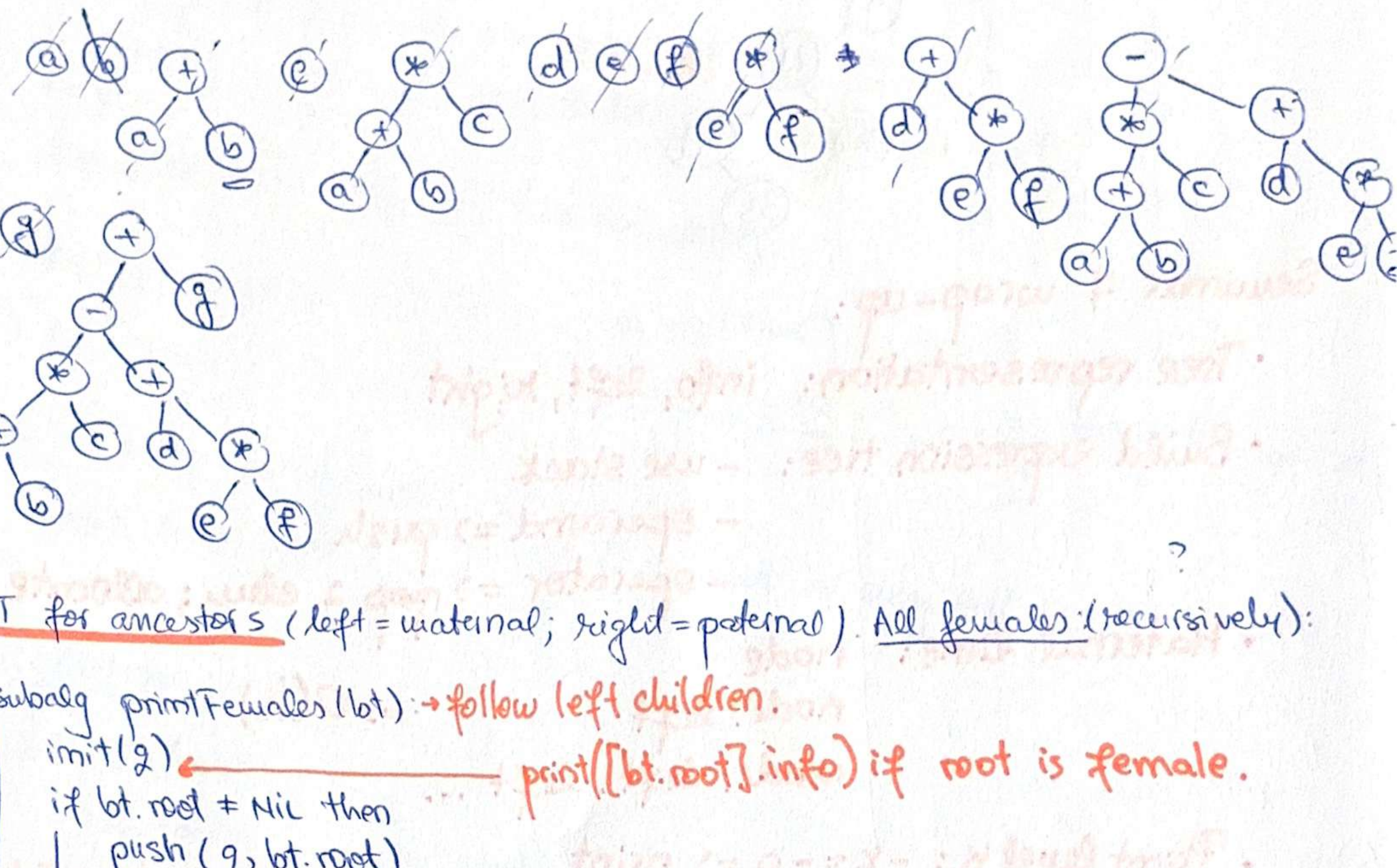
Seminars redone

SCH7

1) BT for expression: O(n)

$(a+b)*c-(d+e*f)+g \Rightarrow$ postfix: $\overbrace{ab+}\overbrace{c*}\overbrace{d\overbrace{ef*}+}-\overbrace{g+}$

Stack:



2) BT for ancestors (left = maternal; right = paternal). All females (recursively):

```
subalg printFemales(bt):  → follow left children.
    init(q)  ←— print([bt.root].info) if root is female.
    if bt.root ≠ Nil then
        push(q, bt.root)
    while not isEmpty(q) ex:                    O(h)
        current ← pop(q)                        ← DFS
        if [current].left ≠ Nil then
            print([[current].left].info)
            push(q, [current].left)
        if [current].right ≠ Nil then
            push(q, [current].right)
```

All ancestors (of degree k) (decrement k)

```
subalg levelK(node, k):
    if node ≠ Nil then
        if k = 0 then
            print([node].info)
        else
            levelK([node].left, k-1)
            levelK([node].right, k-1)
```

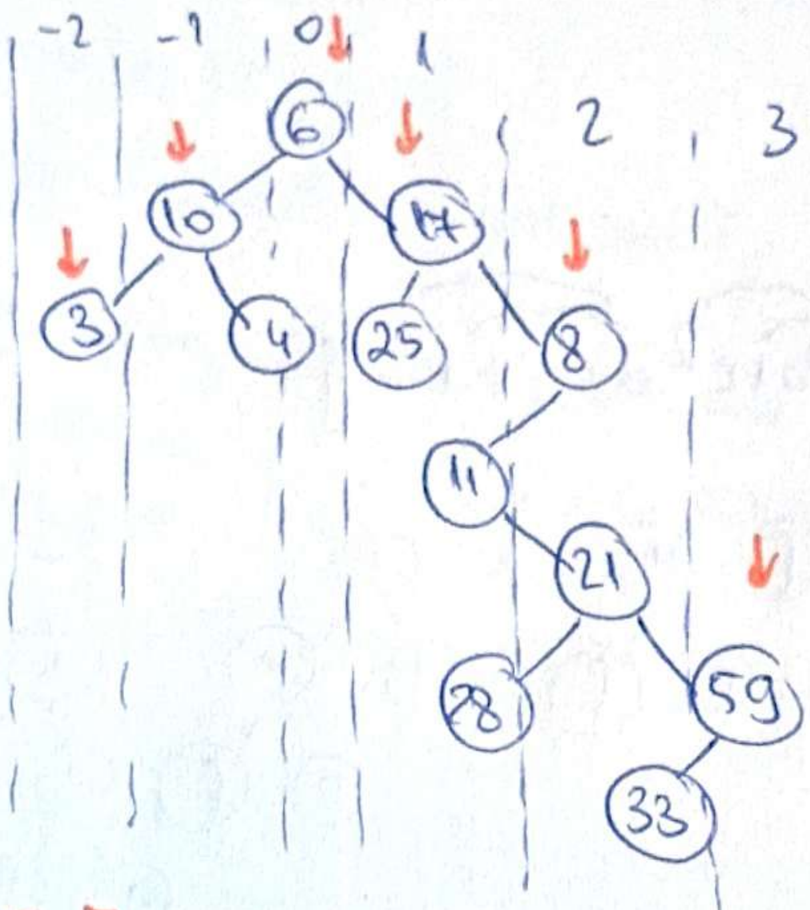
```
subalg printLevelK(bt, k):
    levelK(bt.root, k)
```

O(n)

$O(2^k)$ → level k → at most 2^k nodes

$O(\min(n, 2^k))$

3) top view = nodes visible from top of the tree



=> 3, 10, 6, 17, 8, 59

=> queue of (node, col)

=> map : column → first node found

Seminar 7 wrap-up:

- Tree representation: info, left, right
- Build expression tree: - use stack
  - operand => push
  - operator => pop 2 elem; allocate, push
- Maternal line: node
  node.left -> O(h)
  node.left.left ...
- Print level K: - K == 0 => print
  - else recurse with K-1 => O(min(n, 2^K))
- Height of a tree: 1 + max(left, right) => O(n)
- Top view: - BFS + (node, col)
  - first node in each col. => O(n)

SEM6

1) Iterator for a sorted map on HT, collision res. sep. chaining.

→ hash by KEY only
→ keep each collision list sorted.

Method1: merge all chains into one sorted list. | $\Theta(nm)$

n elems. } => $\frac{n}{m} = \alpha$ = load factor = avg. length of a list
m pos. }

list 1 + list 2 => list 12 => (2α complexity)
⋮

(+) list 1 ... + list m => list 1,2,...,m => $(m\alpha)$

=> $2\alpha + 3\alpha + \dots = \alpha(2+3+\dots) \simeq \alpha \cdot \frac{m(m+1)}{2} = \frac{n}{m} \cdot \frac{m(m+1)}{2} \in \Theta(n \cdot m)$

Method2: merge using heap : $O(n \log m)$

→ min heap of nodes from the first node of every list.

---

2) BD Map

I. 2 hash tables:

key-table	value-table
23 → ball	ball → 23

II. STORE PAIR ONCE.

Node: